What happens if you alias a method and then redefine it? You can try this with a simple program:

class_definitions/wrapper_around_alias.rb
```
class String
  alias_method :real_length, :length

  def length
    real_length > 5 ? 'long' : 'short'
  end
end

"War and Peace".length      # => "long"
"War and Peace".real_length # => 13
```

The previous code redefines String#length, but the alias still refers to the original method. This gives you insight into how method redefinition works. When you redefine a method, you don't really change the method. Instead, you define a new method and attach an existing name to that new method. You can still call the old version of the method as long as you have another name that's still attached to it.

This idea of aliasing a method and then redefining it is the basis of an interesting trick—one that deserves its own example.

The Thor Example

Thor is a Ruby gem for building command-line utilities. Some versions of Thor include a program named rake2thor that converts Rake build files to Thor scripts. As part of doing that, rake2thor must load a Rakefile and store away the names of all the files that are in turn required from that Rakefile. Here is the code where the magic happens:

gems/thor-0.17.0/bin/rake2thor
```
input  = ARGV[0] || 'Rakefile'
$requires = []

module Kernel
  def require_with_record(file)
    $requires << file if caller[1] =~ /rake2thor:/
    require_without_record file
  end
  alias_method :require_without_record, :require
  alias_method :require, :require_with_record
end

load input
```